

Week 4 Tutorial

Welcome to the Week 4 Tutorial. Please follow these instructions step by step - your tutors will also go through them one by one in class. Please ask for their assistance if you're stuck at any place - we are here to help. Today, you will learn the following:

- For loops - short notation for repetitive tasks
- For loops - using iterators for formatting
- For loops and arrays - iterating through an array
- Review of everything up to date- templating an object and using it in an array

Setup

For these tutorials, we will be using an expanded utils.js file. We have added more useful functions to it. You may have created some or all of these in your quiz. Here is an up-to-date utils.js file to work with. We will include this in all our examples in today's tutorial.

```
// A function to create a colour string from 3 inputs
function makeSimpleRGB(inputRed, inputGreen, inputBlue){
  return `rgb(${inputRed}, ${inputGreen}, ${inputBlue})`;
}

// A function to create a random rounded value
function randomRoundedValue(maxValue){
  return Math.round(Math.random() * maxValue);
}

// A function to create a random colour
function makeRGB(redInputValue, greenInputValue, blueInputValue){
  // Let's check if there are values assigned to the input and
  // if not we will give them defaults. The default values will be random ints between 0 and
  255
  let redOutputValue = redInputValue ?? randomRoundedValue(255);
  let greenOutputValue = greenInputValue ?? randomRoundedValue(255);
  let blueOutputValue = blueInputValue ?? randomRoundedValue(255);
  // Now let's make a string out of the output values
  return `rgb(${redOutputValue}, ${greenOutputValue}, ${blueOutputValue})`;
}

// A function to create a rectangle with some default values
function createRect(inputXPos, inputYPos, inputWidth, inputHeight, inputR, inputG, inputB, max
Width = 600, maxHeight = 800){
  let outputXPos = inputXPos ?? randomRoundedValue(maxWidth);
  let outputYPos = inputYPos ?? randomRoundedValue(maxHeight);
  let outputWidth = inputWidth ?? randomRoundedValue(maxWidth);
  let outputHeight = inputHeight ?? randomRoundedValue(maxHeight);
  let outputColour = makeRGB(inputR, inputG, inputB);

  let newRectangle = document.createElementNS("http://www.w3.org/2000/svg", "rect");
  newRectangle.setAttribute("x", outputXPos);
  newRectangle.setAttribute("y", outputYPos);
  newRectangle.setAttribute("width", outputWidth);
  newRectangle.setAttribute("height", outputHeight);
  newRectangle.setAttribute("fill", outputColour);
}
```

```

    return newRectangle;
}

// A function to create a circle with some default values
function createCircle(inputXPos, inputYPos, inputRadius, inputR, inputG, inputB, maxWidth = 600, maxHeight = 800){
    let outputXPos = inputXPos ?? randomRoundedValue(maxWidth);
    let outputYPos = inputYPos ?? randomRoundedValue(maxHeight);
    let outputRadius = inputRadius ?? randomRoundedValue(maxHeight / 2);
    let outputColour = makeRGB(inputR, inputG, inputB);

    let newCircle = document.createElementNS("http://www.w3.org/2000/svg", "circle");
    newCircle.setAttribute("cx", outputXPos);
    newCircle.setAttribute("cy", outputYPos);
    newCircle.setAttribute("r", outputRadius);
    newCircle.setAttribute("fill", outputColour);

    return newCircle;
}

// A function to create an ellipse with some default values
function createEllipse(inputXPos, inputYPos, inputXRadius, inputYRadius, inputR, inputG, inputB, maxWidth = 600, maxHeight = 800){
    let outputXPos = inputXPos ?? randomRoundedValue(maxWidth);
    let outputYPos = inputYPos ?? randomRoundedValue(maxHeight);
    let outputXRadius = inputXRadius ?? randomRoundedValue(maxWidth / 2);
    let outputYRadius = inputYRadius ?? randomRoundedValue(maxHeight / 2);
    let outputColour = makeRGB(inputR, inputG, inputB);

    let newEllipse = document.createElementNS("http://www.w3.org/2000/svg", "ellipse");
    newEllipse.setAttribute("cx", outputXPos);
    newEllipse.setAttribute("cy", outputYPos);
    newEllipse.setAttribute("rx", outputXRadius);
    newEllipse.setAttribute("ry", outputYRadius);
    newEllipse.setAttribute("fill", outputColour);

    return newEllipse;
}

// A function to create a line with some default values
function createLine(inputX1Pos, inputY1Pos, inputX2Pos, inputY2Pos, inputStrokeWidth, inputR, inputG, inputB, maxWidth = 600, maxHeight = 800){
    let outputX1Pos = inputX1Pos ?? randomRoundedValue(maxWidth);
    let outputY1Pos = inputY1Pos ?? randomRoundedValue(maxHeight);
    let outputX2Pos = inputX2Pos ?? randomRoundedValue(maxWidth);
    let outputY2Pos = inputY2Pos ?? randomRoundedValue(maxHeight);
    let outputStrokeWidth = inputStrokeWidth ?? randomRoundedValue(5);
    let outputColour = makeRGB(inputR, inputG, inputB);

    let newLine = document.createElementNS("http://www.w3.org/2000/svg", "line");
    newLine.setAttribute("x1", outputX1Pos);
    newLine.setAttribute("y1", outputY1Pos);
    newLine.setAttribute("x2", outputX2Pos);
    newLine.setAttribute("y2", outputY2Pos);
    newLine.setAttribute("stroke-width", outputStrokeWidth);
    newLine.setAttribute("stroke", outputColour);

    return newLine;
}

// A function to create a rounded rectangle with some default values
function createRoundedRect(inputXPos, inputYPos, inputWidth, inputHeight, inputRoundedness, inputR, inputG, inputB, maxWidth = 600, maxHeight = 800){
    let outputXPos = inputXPos ?? randomRoundedValue(maxWidth);
    let outputYPos = inputYPos ?? randomRoundedValue(maxHeight);
    let outputWidth = inputWidth ?? randomRoundedValue(maxWidth);
    let outputHeight = inputHeight ?? randomRoundedValue(maxHeight);
    let outputRoundedness = inputRoundedness ?? randomRoundedValue(20);
    let outputColour = makeRGB(inputR, inputG, inputB);

    let newRoundedRect = document.createElementNS("http://www.w3.org/2000/svg", "rect");
    newRoundedRect.setAttribute("x", outputXPos);

```

```

    newRoundedRect.setAttribute("y", outputYPos);
    newRoundedRect.setAttribute("width", outputWidth);
    newRoundedRect.setAttribute("height", outputHeight);
    newRoundedRect.setAttribute("rx", outputRoundedness);
    newRoundedRect.setAttribute("ry", outputRoundedness);
    newRoundedRect.setAttribute("fill", outputColour);

    return newRoundedRect;
}

```

We will use the same basic HTML file for all the parts of this tutorial as well.

```

<html lang="en">
  <head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <title>IDEA9103, Week4</title>
  </head>
  <body>
    <svg id="svg" width="800" height="600"></svg>
    <script src="utils.js"></script>
    <script src="script.js"></script>
  </body>
</html>

```

Let's get started with the tasks for today.

1. For loops - short notation for repetitive tasks

OBJECTIVE: Use a 'for' loop to programmatically create and layer 10 rectangles and circles within an SVG element.

CHALLENGE: Understand the structure of the 'for' loop, and the initialisation, condition, and increment statements. Understand how the utils.js file is being used and linked.

DESIRED RESULT: The loop will create 10 rectangles and circle elements and add them to the svg element.

SOLUTION: Implement a 'for' loop that calls the functions included in the utils.js file.

IMPLEMENTATION: The loop will iterate 10 times, and during each iteration, it will generate both a rectangle and a circle. These shapes will be added to the SVG in an alternating pattern: rectangle, circle, rectangle, circle, and so on. Understanding 'for' loops is essential as they allow us to efficiently repeat actions multiple times, making our code more concise and our designs more dynamic.

```

// As always let's grab the svg element from the HTML
let svg = document.getElementById("svg");

// Let's use a for loop to create 10 rectangles and circles.
// The loop uses an iterator i that starts at zero.
// Each time the loop runs we perform the code inside the curly braces and then add 1 to i.
// The loop will run as long as i is less than 10.

for(let i = 0; i < 10; i++){
  let newRect = createRect();
  let newCircle = createCircle();
  // Note the order that we append the elements matters, we will have layers of rectangle, c
  ircle, rectangle, circle, etc.
  svg.appendChild(newRect);

```

```
svg.appendChild(newCircle);  
}
```

2. For loops - using iterators for formatting

We can also utilise the iterator variable in the 'for' loop to make calculations for our shapes. In this exercise, we'll draw 10 rectangles of the same size in a single row.

OBJECTIVE: Create 10 rectangles in a row that are evenly spaced apart.

CHALLENGE: Each rectangle is created using an absolute position. An algorithm is needed that calculates the starting position of each rectangle, so that their spacing is even.

DESIRED RESULT: The 10 rectangles should be evenly spaced in a row.

SOLUTION: To ensure the rectangles are evenly spaced, we'll define variables for the rectangle's size (width and height), spacing along the x axis and the y position. Each rectangle will be spaced apart by the value of `rectSpacing`, and they will all start this distance from the left. As the loop iterates, the x position of each subsequent rectangle is calculated by adding the initial offset and another count of `rectSpacing` plus `rectWidth`.

IMPLEMENTATION: The following code implements the 'for' loop and algorithm required for spacing the rectangles. This demonstrates how to use loops and calculations to create organised and dynamic designs.

```
let svg = document.getElementById("svg"); // Get the svg element  
  
// We can also use the iterator variable in the for loop to make calculations for our shapes  
  
// Let's draw 10 rectangles of the same size in a single row.  
// To make them evenly spaced we need to specify the size, position and spacing using variable  
s  
  
let rectWidth = 50;  
let rectHeight = 50;  
let rectSpacing = 20;  
let rectYPos = 20;  
  
for(let i = 0; i < 10; i++){  
    // Each rectangle will be spaced rectSpacing from each other and they will all start rectS  
pacing from the left  
    // Every iteration we need to add the initial offset and another count of rectSpacing + re  
ctWidth to calculate  
    // the x position of the next rectangle  
    let newRect = createRect(rectSpacing + (i * (rectWidth + rectSpacing)), rectYPos, rectWidt  
h, rectHeight, null, null, null);  
    svg.appendChild(newRect);  
}
```

3. For loops and arrays - iterating through an array

In this exercise, we use a 'for' loop to connect points stored in two arrays to form a shape.

OBJECTIVE: Draw a shape using a series of lines whose coordinates are defined in two arrays, one for the x-coordinates and the other for the y-coordinates.

CHALLENGE: The x-y coordinates in the arrays form a closed loop. To ensure the loop can be closed properly, we need to modify the 'for' loop's behaviour when reaching the last element in the array. If we don't, we risk accessing an element outside the bounds of the array.

DESIRED RESULT: A shape is formed from a series of lines that form a closed loop.

SOLUTION: To ensure we don't encounter errors when reaching the array's end, we employ an 'if-else' statement. This allows us to loop back and connect the shape's last point to its starting point, completing the figure.

IMPLEMENTATION: The following code draws a series of lines as part of a 'for' loop using the x-y coordinates stored in `allX` and `allY`. An 'if-else' statement detects when the last element in the array is reached, and the line is drawn from the the final x-y coordinate in the arrays to the first x-y coordinate.

```
let svg = document.getElementById("svg"); // Get the svg element
svg.setAttribute("width", 600); // Set the width of the svg element
svg.setAttribute("height", 800); // Set the height of the svg element

// Here are 2 arrays that contain the x and y coordinates of the vertices of a shape
let allX = [300, 300, 440, 440, 300, 170, 170, 245, 300, 300, 300, 170, 170, 300, 440, 440, 375, 300];
let allY = [200, 340, 460, 600, 770, 600, 475, 420, 475, 610, 475, 345, 200, 30, 200, 345, 404, 340];

// Here is the for loop that will create the shape
for (let i = 0; i < allX.length; i++) {
  // We need to check where we are in the array using an if statement
  // otherwise we will access an element outside the bounds of the array
  if (i < allX.length - 1) {
    svg.appendChild(createLine(allX[i], allY[i], allX[i + 1], allY[i + 1], 3, 0, 0, 0));
  } else {
    svg.appendChild(createLine(allX[i], allY[i], allX[0], allY[0], 3, 0, 0, 0));
  }
}
```

4. Objects - templating an object and using it in an array

In this exercise, we create a template for a 'Person' object and using this template, we create an array of 'people'. This exercise blends object-oriented programming with some simple visual data representation.

OBJECTIVE: Create a row of circles that are coloured and labelled based on the properties stored in an array of objects.

CHALLENGE: We want a way to have multiple objects that have the same properties but with different values for each of those properties.

DESIRED RESULT: We will create a Person object template that can be used to create or instantiate multiple objects. Each object will have its own set of property values. Those values will

then be shown visually as circles of different colours with text displayed on top of them. The colours and text will be derived from the objects' properties.

SOLUTION: An object template called 'newPerson' will be defined using a function that returns an object. This function will be used to create multiple person objects, each with different property values. A 'for' loop will print the first names for each person object to the console. A second 'for' loop will concatenate all the names together into a single string. A third 'for' loop will create circles that use the colour from the eyeColour property and display text using the firstName, lastName, and age properties.

IMPLEMENTATION: The following code begins with the object-creating function. A 'person1' object is created, followed by a 'people' array containing three person objects. Then, the three 'for' loops perform the tasks described above.

```
// This function creates and returns a new person object with the given properties
function newPerson(firstName, lastName, age, eyeColour) {
  return {
    firstName: firstName,
    lastName: lastName,
    age: age,
    eyeColour: eyeColour
  };
}

// We can now create a person object using the template, to do this we use the new keyword
let person1 = newPerson("John", "Doe", 50, "blue"); // Create a person object
console.log(typeof person1);
console.log("ok");

// We can also create an array of people
let people = [newPerson("John", "Doe", 50, "blue"), newPerson("Jane", "Doe", 40, "green"), newPerson("Jack", "Doe", 30, "brown")]; // Create an array of people

// Like any array we can iterate through the array of objects using a for loop
for (let i = 0; i < people.length; i++) { // Loop through the array
  let person = people[i]; // Get the current person
  console.log("Hello " + person.firstName); // Print the first name of the current person
}

// We can also create a string of all the names
let allNames = ""; // Create a variable to hold all the names
for (let i = 0; i < people.length; i++) { // Loop through the array
  let person = people[i]; // Get the current person
  allNames += person.firstName + " "; // Add the first name of the current person to the allNames variable
}
console.log("Hello " + allNames); // Print all the names at once

let svg = document.getElementById("svg"); // Get the svg element

// Iterate through the array of people and create a circle for each person, in the centre of the circle write their name and age
for (let i = 0; i < people.length; i++) { // Loop through the array
  // Use the createCircle function to create a circle for the current person
  let circle = createCircle(100 + i * 120, 100, 50); // Create a circle
  circle.setAttribute("fill", people[i].eyeColour); // Set the fill colour of the circle - this is going to override the random colour that was made for the circle
  svg.appendChild(circle); // Add the circle to the svg element

  let text = document.createElementNS("http://www.w3.org/2000/svg", "text"); // Create a text element
  text.setAttribute("x", 100 + i * 120); // Set the x position of the text element
  text.setAttribute("y", 100); // Set the y position of the text element
  text.setAttribute("text-anchor", "middle"); // Set the text anchor to the middle
}
```

```
text.setAttribute("fill", "white"); // Set the fill colour of the text element
text.innerHTML = people[i].firstName + " " + people[i].lastName + " " + people[i].age;
// Set the text of the text element
svg.appendChild(text); // Add the text element to the svg element
}
```

5. Inspiration for the week

The inspiration for this week is Argentinian visual artist and creative coder, Manolo Gamboa Naon.

Manolo's works rely on simple geometry but by using a wide colour palette, randomness and iteration (i.e. 'for' loops) he is able to produce interesting works that have complex details and give the feeling of movement.

You can browse Manolo's works via his Behance profile:

<https://www.behance.net/manoloide>  (<https://www.behance.net/manoloide>)

Copyright © The University of Sydney. Unless otherwise indicated, 3rd party material has been reproduced and communicated to you by or on behalf of the University of Sydney in accordance with section 113P of the Copyright Act 1968 (Act). The material in this communication may be subject to copyright under the Act. Any further reproduction or communication of this material by you may be the subject of copyright protection under the Act. Do not remove this notice.

Live streamed classes in this unit may be recorded to enable students to review the content. If you have concerns about this, please visit our [student guide \(https://canvas.sydney.edu.au/courses/4901/pages/zoom\)](https://canvas.sydney.edu.au/courses/4901/pages/zoom) and contact the unit coordinator.

[Privacy Statement \(https://www.sydney.edu.au/privacy-statement.html\)](https://www.sydney.edu.au/privacy-statement.html)